CLIENT DELIVERY / AS-BUILT SPECIFICATION

MEDSYS Backend Client Specification

Formal client-facing specification of the MEDSYS backend as implemented on March 11, 2026, including business behavior, technical architecture, schema, controls, and verification coverage.

DOCUMENT TYPE	SYSTEM
Current Implementation Specification	MEDSYS API Enterprise Backend
VERSION	**DATE**
2.0	March 11, 2026
PREPARED FOR	**BASIS**
Client Review and Delivery	Implemented repository state and verified backend behavior

This document describes the implemented backend only. Operational items not yet verified in a live AWS environment are identified as delivery boundaries.

1. Document Control

VERSION	DATE	OWNER	CHANGE SUMMARY
1.0	March 7, 2026	Backend Delivery Team	Initial as-built specification created from the implemented MVP baseline.
2.0	March 11, 2026	Backend Delivery Team	Consolidated backend implementation, security, testing, schema, and phase outputs into one maintained client document.

2. Introduction

MEDSYS is a healthcare backend platform that provides organization-scoped services for patient registration, appointments, consultations, prescriptions, inventory, analytics, auditing, search, and operational security.

2.1 Purpose

This document provides a single source of truth for what the backend currently delivers.

2.2 Aims and Objectives

- Describe current backend capabilities in business and technical terms.
- Record implemented functional and non-functional requirements.
- Summarize the stack, schema, controls, and verification coverage.
- Provide clients with an auditable delivery baseline.

2.3 Scope

- API behavior exposed through the implemented `/v1/...` surface.
- Worker-based asynchronous audit processing.
- Implemented schema, tests, and operational controls.

2.4 Out of Scope

- Commercial terms and support arrangements.
- Unverified live AWS staging outcomes for Phase 6 rollout.
- Frontend implementation details beyond backend-aligned interfaces.

> **Current delivery status:** backend application functionality is implemented through Phases 1, 2, 3, 4, 5, and 7. Phase 6 infrastructure code exists, but live staging verification remains pending.

3. Executive Summary

The backend is delivered as a TypeScript monorepo with a Fastify API, a dedicated worker service, PostgreSQL persistence, Flyway migrations, Redis-backed asynchronous audit transport, OpenSearch-backed search with database fallback, and observability plus security hardening controls.

Business Modules

- Authentication and access control

Operational Characteristics

- Organization-scoped records and permissions

- Patients and families
- Appointments and encounters
- Prescriptions and dispensing
- Inventory and analytics
- Audit, search, and observability

- JWT access and refresh rotation
- Partitioned clinical and audit tables
- Cache invalidation on writes
- Trace, metrics, and PHI-safe logging
- CI-backed migration and integration coverage

4. Technology Stack and Architecture

LAYER	CURRENT TECHNOLOGY
Runtime	Node.js 20, TypeScript
API Service	Fastify with JWT, rate limit, Swagger, and Swagger UI
Worker Service	Node.js worker consuming audit queue messages
Primary Data Store	PostgreSQL 16
Schema Management	Flyway migrations
Query Layer	Drizzle ORM
Queue and Cache	Redis, with in-memory fallbacks for selected controls
Search	OpenSearch with database fallback
Observability	OpenTelemetry API, Prometheus-style metrics, optional Sentry
Infrastructure	Terraform modules for network, data, messaging, search, and app deployment
CI/CD	GitHub Actions for build, test, Flyway validation, and staging deploy definition

4.1 Logical Architecture

- The API exposes versioned REST endpoints for clinical and administrative operations.
- The worker asynchronously persists audit events and handles retry/DLQ flows.
- PostgreSQL is the system of record for clinical and administrative transactions.
- Redis supports messaging, retries, lockout state, and selected cache surfaces.
- OpenSearch supports patient search acceleration when configured.

5. Roles and Access Model

ROLE	PRIMARY FUNCTION	CURRENT ACCESS SCOPE
Owner	Administrative governance	Full access to implemented modules, including user management, inventory administration, audit logs, analytics, and clinical operational surfaces.
Doctor	Clinical care delivery	Access to patients, profiles, encounters, diagnoses, prescriptions, analytics, and ICD-10 lookup. Appointment creation remains restricted by current backend policy.
Assistant	Front-desk and operational support	Patient registration, family assignment, appointment creation, vitals/timeline support data, inventory operations, and prescription dispensing.

6. API Capability Summary

MODULE	IMPLEMENTED ENDPOINTS	TYPICAL ROLES	CURRENT BEHAVIOR
Authentication	`/v1/auth/login`, `refresh`, `logout`, `register`, `me`, `status`	Public / Authenticated	Identity authentication, session status, token issuance, rotation, and token-family revocation.
Patients	`/v1/patients`, detail, history, profile, family, allergies, conditions, vitals, timeline	Owner, Doctor, Assistant	Patient master data and related support or clinical context records.
Families	`/v1/families`	Owner, Assistant	Family creation and patient-family relationship management.
Appointments	`/v1/appointments`	Owner, Assistant	Scheduling, filtering, retrieval, and status management.
Encounters	`/v1/encounters`, diagnosis and test sub-resources	Owner, Doctor	Atomic encounter bundle creation from eligible appointments.
Prescriptions	`/v1/prescriptions`, queue, detail, dispense	Owner, Doctor, Assistant	Prescription retrieval and atomic dispensing with inventory deduction.
Inventory	`/v1/inventory`, `movements`	Owner, Assistant	Inventory master data, stock history, and movement management.
Search and Lookup	`/v1/search/patients`, `/v1/clinical/icd10`	Authenticated	Patient search and ICD-10 suggestions.
Analytics	`overview`, `cache`, `observability`	Authenticated	Operational counts, cache stats, and observability summaries.
Audit	`/v1/audit/logs`	Owner	Filterable audit retrieval backed by async persistence.
Operational	`/healthz`, `/metrics`, `/api/v1/docs`	Mixed	Health checks, metrics export, and OpenAPI documentation.

7. Functional Requirements

ID	AREA	IMPLEMENTED REQUIREMENT
FR-001	Authentication	Authenticate users by email, password, and organization context.
FR-002	Authentication	Reject invalid credentials and inactive users.
FR-003	Tokens	Issue RS256 access tokens and persist refresh tokens.
FR-004	Session Security	Rotate refresh tokens, track token families, and revoke families on replay detection.
FR-005	Authentication	Expose session status, authenticated profile lookup, logout, and controlled registration flows.
FR-006	Users	List and create organization users with role assignment.
FR-007	Patients	Create, list, retrieve, update, and soft-delete patient records.
FR-008	Patients	Validate NIC, date of birth, age consistency, and compatible patient-name input formats.
FR-009	Patient Profile	Provide allergies, conditions, vitals, timeline, and note-based history.
FR-010	Families	Create family records and assign patients through family membership links.
FR-011	Appointments	Create, list, retrieve, filter, and update appointments.
FR-012	Encounters	Create encounter bundles only from eligible appointments.
FR-013	Encounters	Save encounters, diagnoses, tests, optional prescriptions, and appointment completion atomically.
FR-014	Encounters	Enforce one encounter per appointment.
FR-015	Prescriptions	Create, list, and retrieve prescriptions with items and dispense history.
FR-016	Dispensing	Expose a pending dispense queue for prescriptions not yet dispensed.
FR-017	Dispensing	Prevent duplicate dispensing and verify stock before deduction.
FR-018	Inventory	Create, list, update, and movement-track inventory items.
FR-019	Analytics	Provide analytics overview counts.
FR-020	Search	Provide patient search and ICD-10 suggestions through backend endpoints.
FR-021	Caching	Cache selected hot reads and invalidate cache entries on relevant writes.
FR-022	Audit	Publish audit events for implemented administrative and clinical actions.
FR-023	Audit Worker	Retry failed audit events with backoff and route exhausted messages to a dead-letter queue.
FR-024	Observability	Expose metrics, request traces, and authenticated observability summaries.

ID	AREA	IMPLEMENTED REQUIREMENT
FR-025	Documentation	Expose a health endpoint and OpenAPI documentation.

8. Non-Functional Requirements

ID	CATEGORY	IMPLEMENTED REQUIREMENT
NFR-001	Security	Use RS256 JWT authentication for protected API access.
NFR-002	Security	Fail startup when mandatory environment variables or RSA key material are missing.
NFR-003	Security	Enforce backend-side permission checks independent of the frontend.
NFR-004	Security	Detect refresh-token replay and revoke token families.
NFR-005	Security	Apply brute-force lockout and sensitive-action throttling controls.
NFR-006	Security	Scrub PHI and secret-bearing values from structured logs and error reports.
NFR-007	Integrity	Preserve transactional integrity for encounter bundle creation and prescription dispensing.
NFR-008	Integrity	Prevent negative stock through implemented write flows.
NFR-009	Reliability	Retain audit events through retry and dead-letter handling.
NFR-010	Reliability	Shut the worker service down cleanly on termination signals.
NFR-011	Scalability	Support separate analytics/reporting reads through `DATABASE_READ_URL`.
NFR-012	Scalability	Partition appointments and audit logs to support growth-oriented storage management.
NFR-013	Maintainability	Manage schema evolution through Flyway migrations with rollback guidance.
NFR-014	Maintainability	Validate migrations and DB-backed flows in CI.
NFR-015	Observability	Provide request metrics, rate/error trends, and traceable request correlation by request ID.
NFR-016	Compliance	Maintain documented control mappings, secrets rotation guidance, and disaster recovery planning artifacts.
NFR-017	Testability	Maintain DB-backed integration coverage for critical clinical flows and seeded-data validity.
NFR-018	Deployment	Include infrastructure-as-code and deployment workflow definitions for staged rollout.
NFR-019	Boundary	Live staging deployment verification remains pending even though infrastructure implementation artifacts are present.

9. Data Model, Tables, and Relationships

TABLE	KEY ATTRIBUTES	RELATIONSHIPS AND NOTES
users	`id`, `organization_id`, `uuid`, `email`, `password_hash`, `first_name`, `last_name`, `role`, `is_active`, timestamps	Core user table. Unique email per organization.
refresh_tokens	`user_id`, `token_id`, `family_id`, `parent_token_id`, `used_at`, `replay_detected_at`, `expires_at`, `revoked_at`	Refresh token lifecycle and replay-protection table. Many-to-one to users.
families	`family_code`, `family_name`, `assigned`, timestamps, `deleted_at`	Family master referenced by patients and family-membership records.
patients	`nic`, names, `dob`, `age`, `gender`, `phone`, `address`, `family_id`, status flags, timestamps	Referenced by appointments, encounters, prescriptions, allergies, conditions, vitals, timeline, and history.
family_members	`family_id`, `patient_id`, `relationship`, timestamps	Join table between families and patients. Unique family/patient pair.
patient_allergies	`patient_id`, `allergy_name`, `severity`, flags, timestamps	Many allergies per patient.
patient_conditions	`patient_id`, `condition_name`, `icd10_code`, `status`, timestamps	Many conditions per patient.
appointments	`patient_id`, `doctor_id`, `assistant_id`, `scheduled_at`, `status`, `priority`, timestamps	Monthly range-partitioned by `scheduled_at`. Composite primary key.
encounters	`appointment_id`, `appointment_scheduled_at`, `patient_id`, `doctor_id`, `checked_at`, `notes`, `next_visit_date`, timestamps	One encounter per appointment via composite foreign key to appointments.
encounter_diagnoses	`encounter_id`, `icd10_code`, `diagnosis_name`, timestamps	Many diagnoses per encounter.
test_orders	`encounter_id`, `test_name`, `status`, timestamps	Many test orders per encounter.
prescriptions	`encounter_id`, `patient_id`, `doctor_id`, timestamps	Prescription header linked to encounter, patient, and doctor.
prescription_items	`prescription_id`, `drug_name`, dose/frequency/duration, `quantity`, `source`	Many items per prescription.
dispense_records	`prescription_id`, `assistant_id`, `dispensed_at`, `status`, `notes`	Dispense event table. Implemented write flow supports one successful dispense outcome.
inventory_items	`sku`, `name`, `category`, `unit`, `stock`, `reorder_level`, flags, timestamps	Inventory master used by dispensing and movement tracking.
inventory_movements	`inventory_item_id`, `movement_type`, `quantity`, `reference_type`, `reference_id`, `created_by_id`, `created_at`	Immutable inventory transaction history.
patient_vitals	`patient_id`, `encounter_id`, blood pressure, heart rate, temperature, `spo2`, `recorded_at`, timestamps	Repeated measurements per patient, optionally linked to encounters.

TABLE	KEY ATTRIBUTES	RELATIONSHIPS AND NOTES
patient_timeline_event	`patient_id`, `encounter_id`, `event_date`, `title`, `description`, `event_kind`, `tags`, `value`, timestamps	Chronological patient narrative events.
patient_history_entries	`patient_id`, `created_by_user_id`, `note`, timestamps, `deleted_at`	Dedicated note-based patient history resource.
audit_logs	`actor_user_id`, `entity_type`, `entity_id`, `action`, `ip`, `user_agent`, `request_id`, `payload`, `created_at`	Monthly partitioned audit store with request correlation.

9.1 Relationship Summary

- One organization contains many users, families, patients, appointments, encounters, prescriptions, inventory items, refresh tokens, and audit logs.
- One family can contain many patients through the `family_members` join table.
- One patient can have many allergies, conditions, vitals, timeline events, history entries, appointments, encounters, and prescriptions.
- One appointment can have at most one encounter.
- One encounter can have many diagnoses and test orders and can produce one prescription header with multiple prescription items.
- One inventory item can have many inventory movement records.

10. Test Coverage and Verification

COVERAGE AREA	CURRENT EVIDENCE	SCOPE
API automated tests	48 passing	Auth, patients, users, analytics, families, appointments, encounters, prescriptions, inventory, audit, validation, and deny-paths.
Worker automated tests	6 passing	Audit queue retry, dead-letter handling, and worker-side persistence.
DB-backed integration tests	Implemented	Appointment to encounter to prescription path, prescription to dispense to inventory movement path, role matrix, and seed validity.
CI gates	Implemented	Flyway validation, migration smoke setup, build, and full test execution.

11. Runtime and Delivery Boundaries

- Mandatory runtime variables include `DATABASE_URL`, `ORGANIZATION_ID`, and JWT RSA key material.
- Optional features such as Redis queueing, OpenSearch acceleration, Sentry reporting, and replica reads depend on runtime configuration.
- Infrastructure-as-code and deployment workflows are implemented, but live AWS staging verification remains pending and is not claimed here as operational acceptance.

This is the consolidated client-facing backend specification for MEDSYS as of March 11, 2026.